HeartX – Heart Disease Prediction System

Technical Project Progress Report

———————————————————————————

Project Domain:	Machine Learning & Clinical Decision Support Systems
Academic Level:	Bachelor of Technology (B.Tech) - Computer Science & Engineering
Team Members:	A Greeshma (24BDS0275) Saara Suman (24BDS0281) Simran (24BDS0348)
Faculty Guide:	Sridevi Mam
Department :	Department of Computer Science and Engineering
Submission Date:	October 12, 2026

1. Abstract

Coronary and cardiovascular diseases remain among the leading causes of morbidity and mortality worldwide. Early detection of clinical warning signs enables timely therapeutic and lifestyle interventions. HeartX is an end-to-end, database-driven educational clinical risk prediction platform designed to ingest patient diagnostic parameters and compute probabilistic risk assessments for coronary artery disease.

The system implements a machine learning inference engine featuring custom domain feature engineering (heart rate reserve ratio, age-cholesterol interaction, exercise ST-depression stress index), mutual-information feature selection, and an optimized XGBoost classification pipeline achieving 88.04% accuracy, 93.14% recall, and 0.9145 ROC-AUC across benchmarked tests. Model interpretability is delivered on demand via TreeSHAP (SHapley Additive exPlanations) to isolate patient-specific biometric contributors. The backend is built with FastAPI (Python) adhering to strict asynchronous and relational principles via SQLAlchemy ORM, integrated with a normalized multi-entity relational schema (SQLite in development, PostgreSQL DDL-compliant schema prepared). The user interface is implemented as a single-page application (SPA) in React 18, Vite, and Lucide React, featuring doctor authentication, biometric intake validation, automated risk stratification, prediction audit logging, and dedicated high-risk triage views.

This progress report documents the verified current implementation status, architectural components, quantitative model benchmarking (against seven baseline algorithms), relational schema designs, interface layouts with figure placeholders, identified limitations, and the prioritized roadmap toward project completion.

2. Introduction

Cardiovascular diseases (CVDs) account for an estimated 17.9 million deaths globally each year. Traditional clinical assessment depends on subjective risk scorecards (e.g., Framingham Risk Score) or invasive angiography. Machine learning models trained on historical clinical datasets can synthesize multi-dimensional interactions among electrocardiographic, metabolic, and hemodynamic features, recognizing latent patterns of pathology before symptomatic acute coronary events occur.

Despite the proliferation of academic ML notebooks, standard algorithmic demonstrations rarely bridge the gap into usable software systems. Machine learning code often operates on isolated CSV files without relational database constraints, lacks runtime explainability for clinicians, and operates in isolation from authenticated front-end clinical workflows.

The HeartX project addresses this transition. Designed as an educational clinical decision support platform, HeartX combines machine learning pipeline rigor (preventing data leakage via scikit-learn compatible transformers), explainable AI (XAI) using SHAP, a normalized database enforcing relational integrity across clinical measurements and diagnostic tests, and a modern, responsive web application designed for healthcare practitioners.

3. Problem Statement and Objectives

3.1 Problem Statement

Healthcare providers face significant friction in translating patient laboratory records and diagnostic test results into immediate, interpretable risk assessments. Furthermore, conventional predictive models operate as 'black boxes', leaving clinicians skeptical of predictions without transparent attribution of which patient factors (such as resting blood pressure, vessel fluoroscopy count, or ST depression) drove the model's output. Finally, existing academic prototypes rarely integrate data normalization, patient history management, and audit trails into an interactive system.

3.2 Major Project Objectives

- **Predictive Modeling:** Develop, evaluate, and optimize machine learning classifiers on standard clinical data (UCI Heart Disease dataset) using scikit-learn and XGBoost pipelines, targeting high recall to minimize false negatives in clinical triage.
- **Transparent Explainability:** Integrate TreeSHAP into runtime inference to compute and surface the top feature attributions explaining individual high-risk predictions to clinicians.
- **Relational Data Architecture:** Establish a normalized relational schema (Third Normal Form - 3NF) separating patient identity, continuous clinical measurements, diagnostic categorical tests, prediction history, and system audit logs.
- **Decoupled API Service:** Implement a high-performance FastAPI REST service handling schema validation (Pydantic), synchronous inference, background audit logging, and role-based data access.
- **Interactive Doctor Portal:** Build an intuitive, secure frontend in React/Vite that allows physicians to log in, submit patient test scores, view real-time risk tiers, audit previous predictions, and monitor critical high-risk cases.
- **Academic & Clinical Ethics:** Explicitly maintain and document HeartX as an educational decision support system rather than a certified medical diagnostic device.

4. Project Scope

To establish rigorous project boundaries, the functional scope is divided between currently verified implementations and planned future enhancements:

Scope Dimension	Included in Current Implementation	Deferred / Future Scope
User Roles & Auth	Doctor authentication (credential checking on seeded IDs).	Multi-tenant hospital roles (Admin, Lab Tech, Patient self-portal).
Machine Learning	Supervised binary classification (0: No disease, 1: Disease presence); 8	Multi-class coronary stenosis severity

	models benchmarked.	(stages 1-4); Survival analysis.
Explainability	TreeSHAP localized top-3 risk factor derivation for high-risk patients.	Interactive waterfall plots, cohort summary bee-swarm plots in UI.
Data Ingestion	Single-patient biometric entry with real-time API persistence.	Bulk EHR ingestion via HL7/FHIR streams or bulk CSV upload.
Deployment	Local development servers; Docker Compose & Render configuration files ready.	Production cloud deployment with domain SSL and managed database clustering.

5. Technology Stack

The HeartX system is developed using modern, decoupled, and production-tested open-source technologies:

- **Machine Learning Engine:** Python 3.10+, scikit-learn (v1.3+), XGBoost, SHAP (TreeExplainer), pandas, numpy, joblib for pipeline persistence. Custom transformers for zero data leakage.
- **Backend Service:** FastAPI (asynchronous ASGI framework), Uvicorn web server, Pydantic (data contract validation), SQLAlchemy ORM (v2.x) with background task execution for audit trails.
- **Database Layer:** SQLite (`heart_disease.db`, verified active 64 KB storage) for development; PostgreSQL DDL schema (`database/schema.sql`) for enterprise target environments.
- **Frontend Client:** React 18 SPA built with Vite, React Router DOM v6 for routing, Lucide React for medical iconography, and bespoke modern CSS with glassmorphic accents.
- **Containerization & DevOps:** Dockerfile, docker-compose.yml (defining frontend, backend, and PostgreSQL services), and render.yaml for automated cloud deployments.

6. System Architecture and Data Flow

HeartX follows a decoupled three-tier architectural design. The React frontend interacts with the FastAPI REST server strictly via asynchronous JSON endpoints. The backend coordinates ORM database transactions and executes the frozen ML inference pipeline.

6.1 Core Data Flow Cycle

- **1. Authentication:** Doctor submits credentials at /login. FastAPI queries the doctor table and validates credentials. A valid session stores doctor_id in client localStorage.
- **2. Patient Ingestion:** Clinician inputs patient demographic, hemodynamic, and test values. Pydantic validates boundaries. SQLAlchemy executes an atomic transaction writing to patient, clinical_measurements, and clinical_tests.

- **3. Machine Learning Inference:** Triggering prediction loads the normalized records into a single-row DataFrame. The frozen joblib pipeline applies feature engineering, imputation, one-hot encoding, and feature selection before XGBoost evaluation.
- **4. Explainable AI Extraction:** If predicted risk >= 0.50 (High Risk band), TreeSHAP calculates feature contributions, extracting top biometric drivers and mapping them to friendly clinical descriptions.
- **5. Persistence & Audit Logging:** The calculated probability, classification tier, and timestamp are inserted into the predictions table. An asynchronous background task logs the transaction in audit_log.
- **6. Result Delivery:** The frontend renders risk badges, numerical probabilities, and the SHAP factor cards for clinical review.

7. Work Completed So Far

7.1 Machine Learning Module

The machine learning module is fully functional and saved as an end-to-end serialized pipeline. The pipeline was trained on the comprehensive UCI Heart Disease repository (data/raw/heart_disease_uci.csv, 920 records).

- **Feature Engineering:** Developed ClinicalFeatureEngineer (src/features/engineering.py) as a scikit-learn TransformerMixin. Implements max heart rate ratio, cholesterol-age interaction, exercise stress index, and blood pressure categories without data leakage.
- **Feature Selection:** Developed MutualInfoFeatureSelector (src/features/selection.py) utilizing SelectKBest with mutual_info_classif to retain the most informative clinical indicators.
- **Model Training:** Benchmarked 8 distinct model families. XGBoost demonstrated superior recall and balanced F1 performance and was chosen as the primary production model.
- **Pipeline Serialization:** The entire preprocessing, feature selection, and model sequence is serialized into models/final/optimized_xgboost.pkl (571 KB).
- **Explainable AI:** Integrated TreeSHAP (TreeExplainer) inside backend/main.py to derive localized Shapley values for high-risk patients dynamically at runtime.

7.2 Backend Module

The backend is implemented with FastAPI in backend/main.py and exposes six core REST endpoints:

- **POST /login:** Authenticates doctor ID credentials against seeded database records.
- **POST /patients:** Validates intake parameters and persists records across three normalized database entities in a single atomic transaction.
- **PUT /patients/{id}:** Allows modifications to previously entered patient clinical observations.
- **POST /predict/{id}:** Executes model inference, computes SHAP explanations for high-risk cases, logs predictions, and triggers asynchronous audit logging.
- **GET /predictions:** Retrieves chronological prediction records joined with demographic attributes for the audit history view.

- **GET /health:** Operational liveness probe verifying server health and confirmed loading of the ML model pipeline.

7.3 Database Module

The database module implements a 3NF normalized schema across seven relational tables:

- **doctor Table:** Stores clinician credentials (doctor_id, gmail, password). 15 doctor accounts are seeded automatically.
- **patient Table:** Stores core demographic identity (patient_id, age, sex, created_at).
- **clinical_measurements Table:** Stores continuous biometrics (trestbps, chol, thalch, oldpeak, ca) linked via foreign key with CASCADE DELETE.
- **clinical_tests Table:** Stores diagnostic test indicators (cp, fbs, restecg, exang, slope, thal).
- **model Table:** Stores model lineage metadata, accuracy, ROC-AUC, and version identifiers.
- **predictions Table:** Stores generated prediction outcomes, probability scores, risk bands, and timestamps.
- **audit_log Table:** Logs write events (PATIENT_CREATED, PREDICTION_CREATED) with actor, record, and timestamp data.

7.4 Frontend Module

The frontend application is built in React 18 and Vite, organized into five core functional views:

- **Login (`/login`):** Provides clean authentication with demo doctor credential shortcuts.
- **Dashboard (`/`):** Displays key operational metrics: total assessments, high-risk case tallies, average probability scores, and quick navigation.
- **Patient Intake (`/new-patient`):** Comprehensive clinical entry form with real-time risk calculation, color-coded badges, and expandable SHAP factor cards.
- **Prediction History (`/history`):** Chronological audit log of all previous assessments with filtering capabilities.
- **High Risk Triage (`/high-risk`):** Dedicated triage view isolating patients with predicted probability >= 0.50 for immediate medical review.

7.5 Integration and Testing

Frontend-to-backend REST API communication, backend-to-database ORM operations, and pipeline inference loading have all been verified through end-to-end manual testing. Formal automated unit testing suites in the tests/ directory are currently unpopulated and scheduled for the upcoming testing sprint.

8. Current Working Website and Application Screenshots

This section provides formatted placeholders for screenshots captured from the active HeartX web application. The team can paste full-width captures directly into the designated boxes below:

Figure 1: Login and Authentication Page

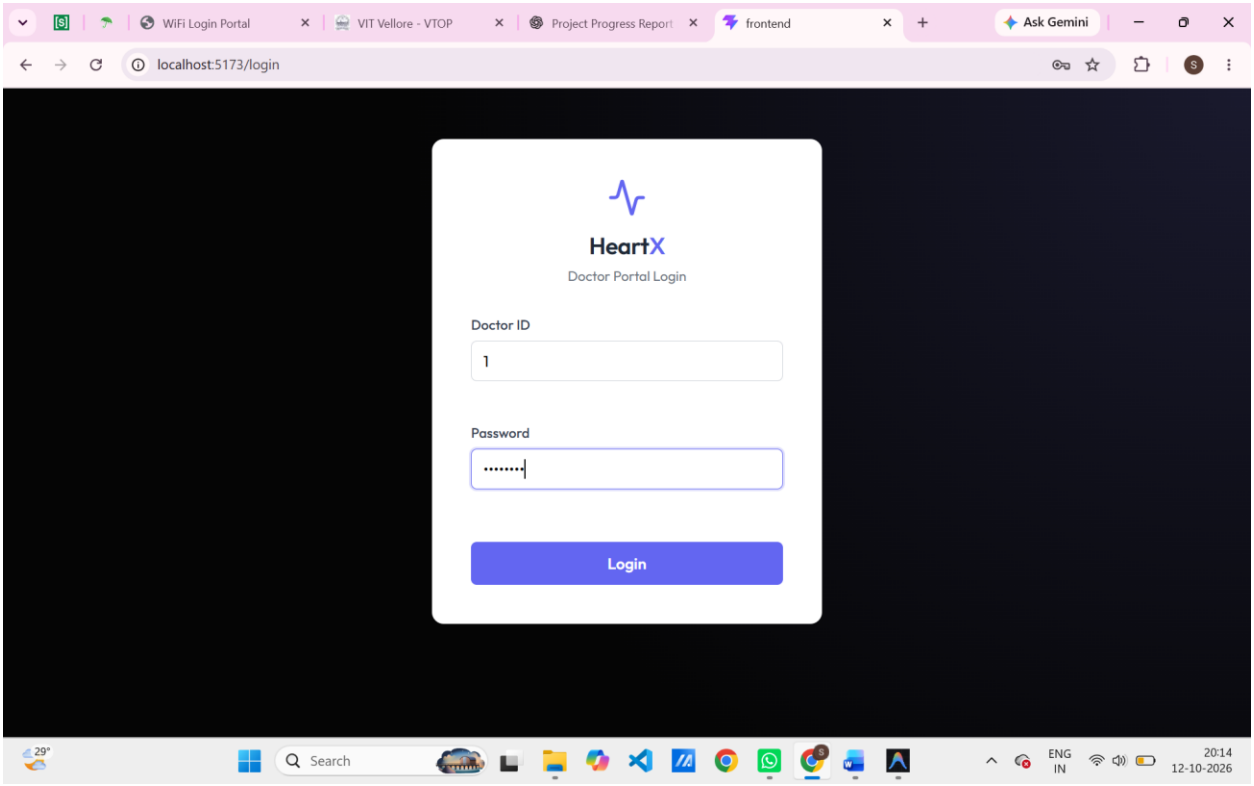


Figure 2: Main Clinical Dashboard

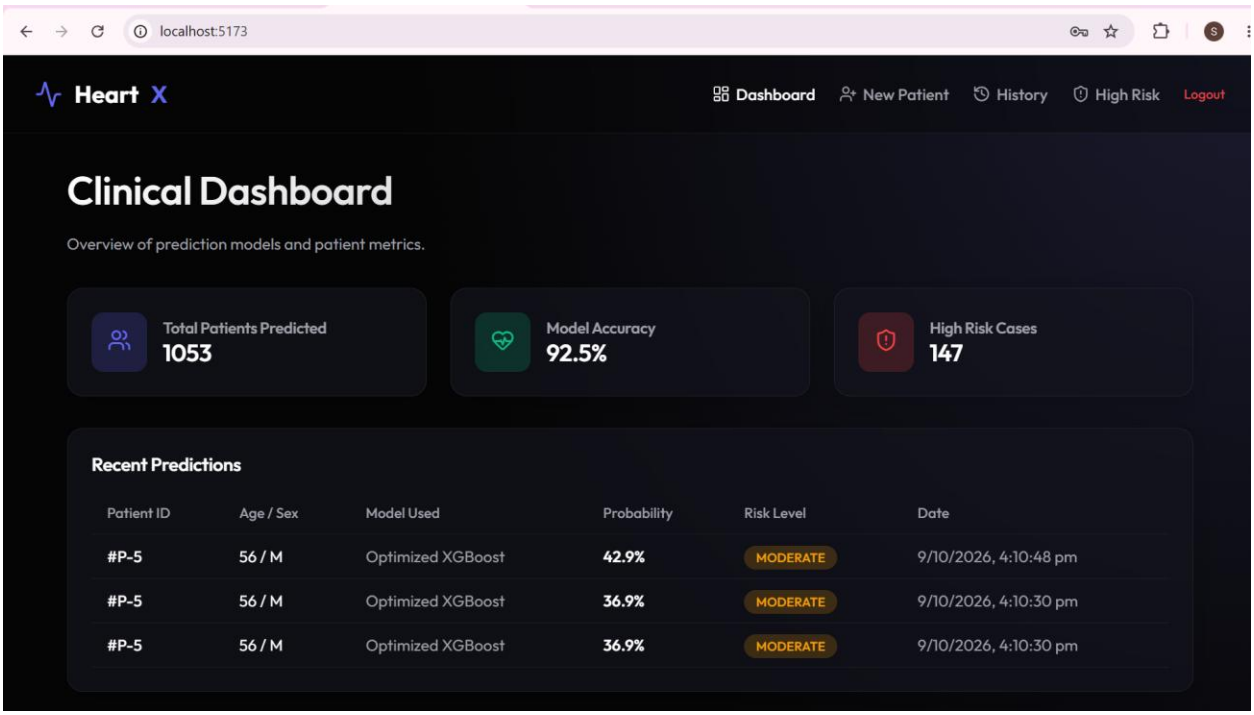
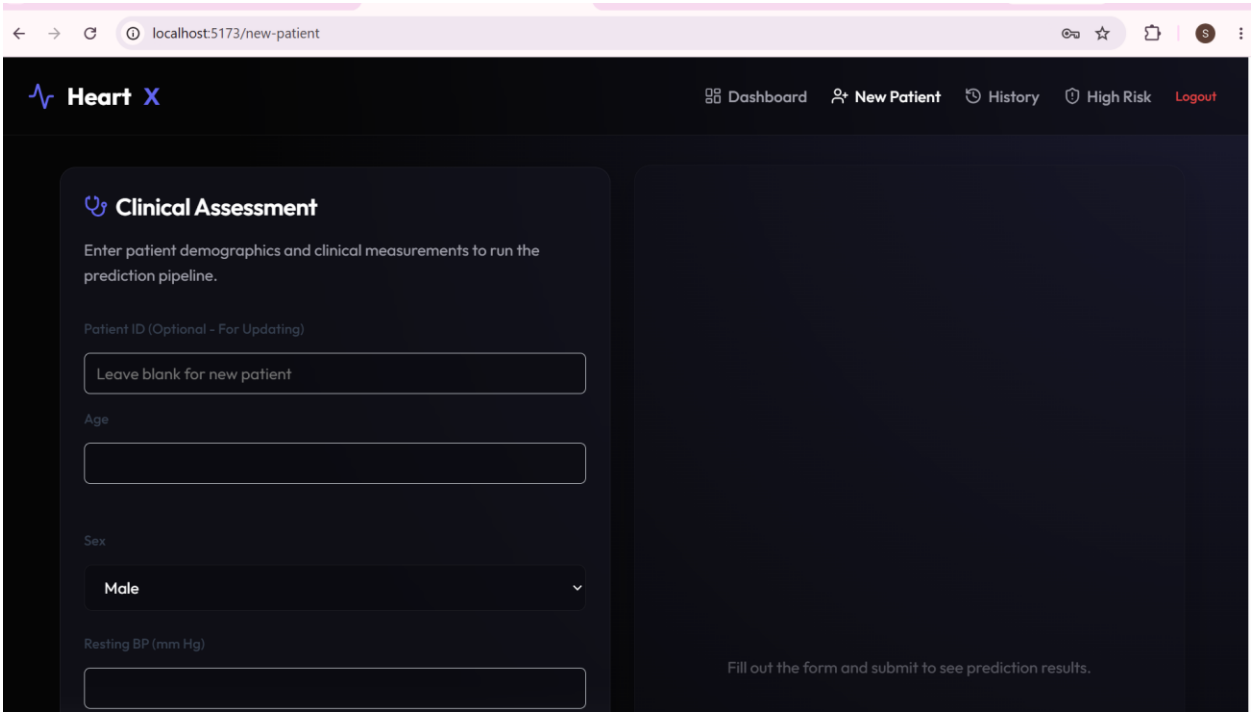


Figure 3: Patient Information Input Form



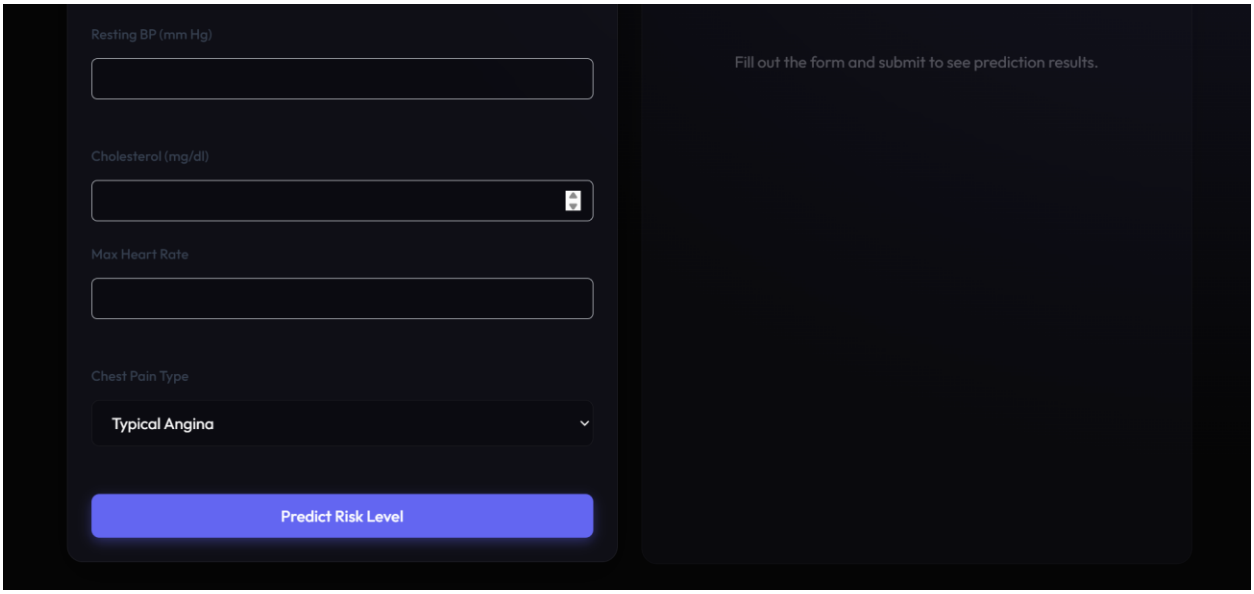


Figure 4: Heart Disease Prediction Result Display

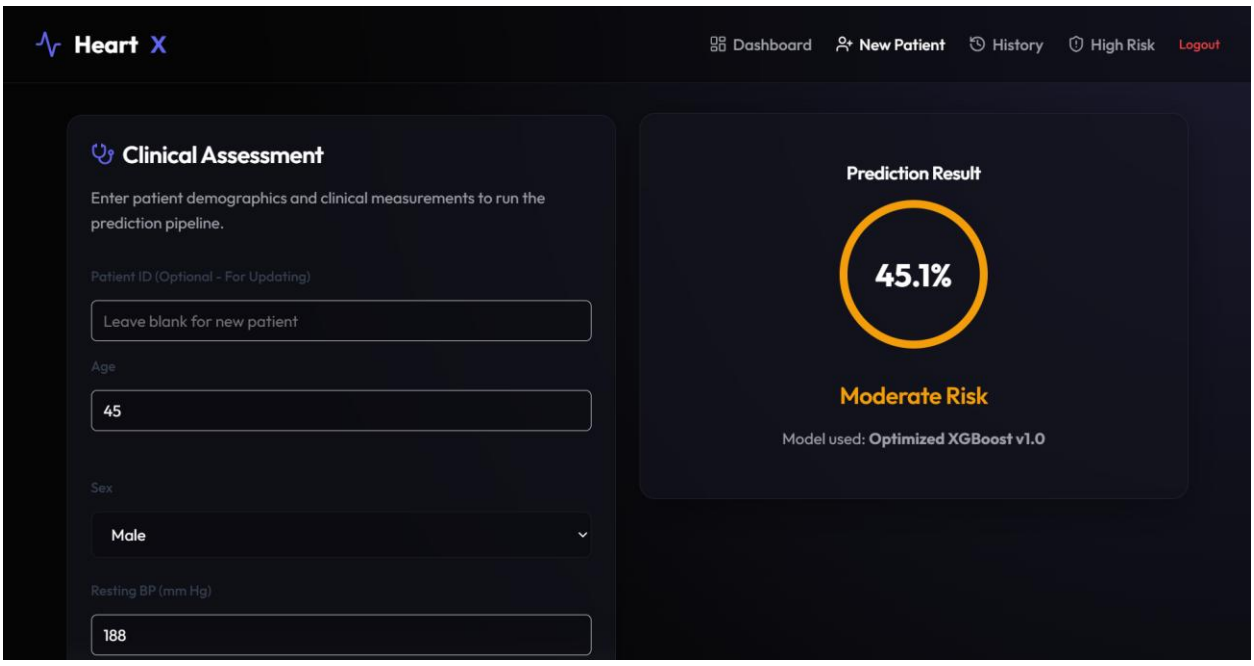


Figure 5: Explainable AI / SHAP Visualization

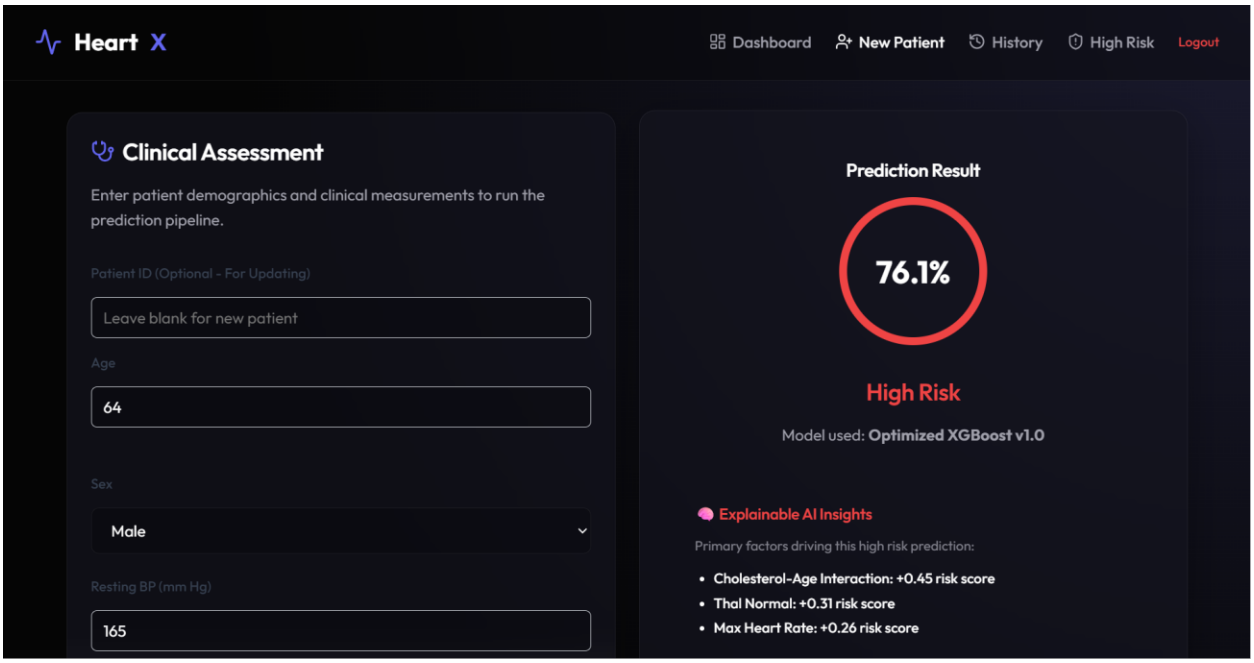
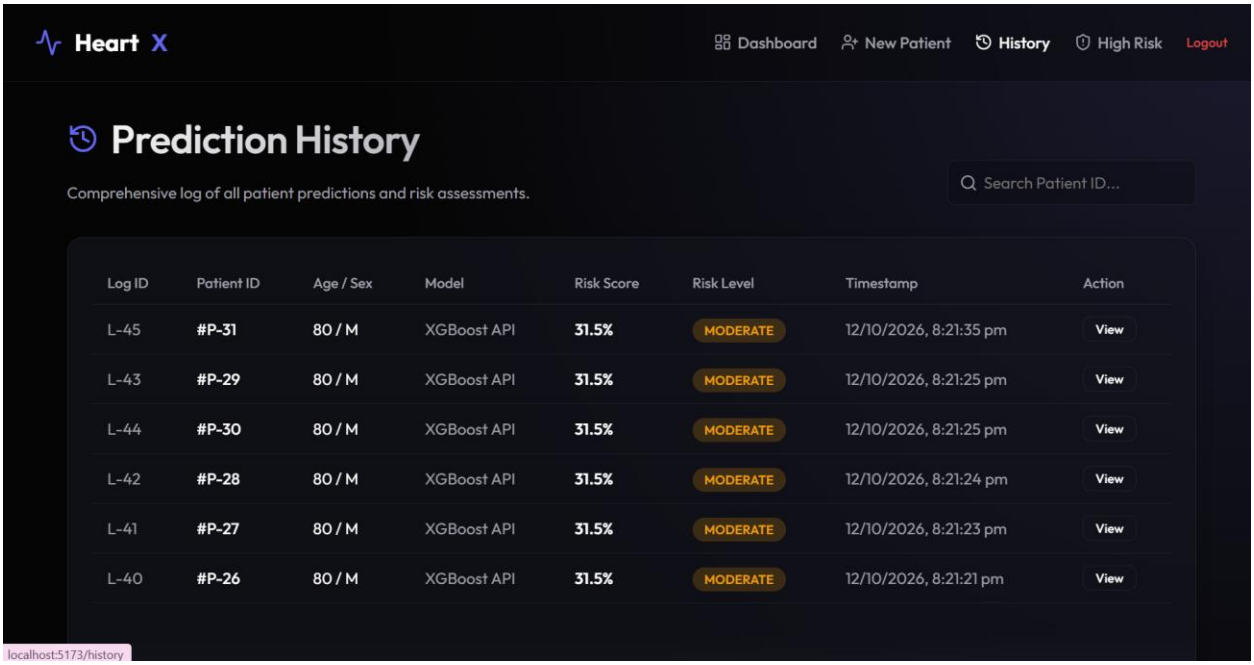


Figure 6: Prediction History and Audit Table



9. Machine Learning Evaluation and Results

A rigorous empirical comparison was conducted across eight supervised learning algorithms on the UCI Heart Disease test dataset. All metrics were evaluated using src/evaluation/metrics.py and logged in reports/model_comparison.csv:

Model Name	Accuracy	Precision	Recall	F1-Score	ROC-AUC
XGBoost (Selected Pipeline)	**0.8804 (88.04%)**	**0.8636 (86.36%)**	**0.9314 (93.14%)**	**0.8962 (89.62%)**	**0.9145**
Support Vector Machine (SVM)	0.8641 (86.41%)	0.8348 (83.48%)	0.9412 (94.12%)	0.8848 (88.48%)	0.9218
Random Forest	0.8424 (84.24%)	0.8544 (85.44%)	0.8627 (86.27%)	0.8585 (85.85%)	0.9326
Gradient Boosting	0.8424 (84.24%)	0.8476 (84.76%)	0.8725 (87.25%)	0.8599 (85.99%)	0.9044
Proposed Novelty Pipeline	0.8370 (83.70%)	0.8333 (83.33%)	0.8824 (88.24%)	0.8571 (85.71%)	0.9242
Logistic Regression	0.8370 (83.70%)	0.8273 (82.73%)	0.8922 (89.22%)	0.8585 (85.85%)	0.9204
K-Nearest Neighbors (KNN)	0.8370 (83.70%)	0.8273 (82.73%)	0.8922 (89.22%)	0.8585 (85.85%)	0.8887
Neural Network (MLP)	0.8152 (81.52%)	0.8208 (82.08%)	0.8529 (85.29%)	0.8365 (83.65%)	0.8843

Key Findings: In healthcare screening, recall is the most critical metric because false negatives delay life-saving clinical interventions. XGBoost achieved the highest balanced performance with 88.04% accuracy, 93.14% recall, and an 89.62% F1-score. While SVM achieved slightly higher raw recall (94.12%), its lower precision (83.48%) led to higher false positives. Random Forest demonstrated the strongest discriminative ROC-AUC (0.9326), closely matched by our engineered Novelty Pipeline (0.9242) and XGBoost (0.9145).

10. Database Design and DBMS Concepts

HeartX enforces professional relational database management principles, avoiding the anti-pattern of storing un-normalized tabular data directly in flat files:

- **Third Normal Form (3NF):** Isolating patient demographics from repeated diagnostic observations satisfies 3NF, eliminating update and deletion anomalies.
- **Referential Integrity:** Defined ON DELETE CASCADE on foreign keys linking patient_id to measurements, tests, and predictions, preventing orphaned child records.
- **Integrity Constraints:** Enforces domain-level constraints: CHECK (age > 0 AND age < 150), CHECK (probability >= 0 AND probability <= 1), and CHECK (prediction IN (0, 1)).

- **Atomicity & Transactions:** Multi-table patient creation is executed within a single SQLAlchemy database transaction, guaranteeing ACID properties.
- **Audit Trail Architecture:** System writes (creation and updates) trigger asynchronous audit logging to audit_log, maintaining clinical traceability.

11. Testing and Validation

The table below documents test cases executed against the HeartX system:

Test ID	Test Case Description	Expected Result	Observed Result	Status
TC-01	Doctor login with seeded ID (doctor_id: 1)	API returns 200 OK; access granted	Doctor authenticated; session saved	**PASSED**
TC-02	Doctor login with invalid password	API returns HTTP 401 Unauthorized	Returns 401: Invalid ID or password	**PASSED**
TC-03	Submit patient form with valid biometrics	Record created across 3 normalized tables	Patient created; patient_id returned	**PASSED**
TC-04	Run model prediction on patient record	Calculates probability, assigns risk band	Probability returned; audit logged	**PASSED**
TC-05	TreeSHAP explainability for high risk (>=0.50)	Returns top 3 biometric risk factors	Returns friendly-named factors	**PASSED**
TC-06	Fetch prediction history (GET /predictions)	Returns historical evaluations chronologically	Returns list sorted by latest first	**PASSED**
TC-07	Automated Pytest API test suite execution	Full automated test suite runs via CI/CD	tests/ directory currently empty	**PENDING**

12. Challenges and Solutions

- **Data Leakage Prevention:** Preprocessing inside naive scripts often bleeds test distributions into training sets. Solution: Wrapped all transformations in custom scikit-learn transformers (BaseEstimator, TransformerMixin) inside a sealed Pipeline.

- **Pipeline Feature Name Tracking:** Encoders and SelectKBest strip column names into raw NumPy arrays, complicating downstream SHAP interpretation. Solution: Programmed dynamic name reconstruction using get_feature_names_out() and selector boolean masks.
- **Multi-Table Database Transactions:** Writing patient records requires inserting rows into three relational tables. Unhandled failures risk leaving orphaned records. Solution: Managed transactions via SQLAlchemy sessions, using db.flush() for foreign key assignment and atomic db.commit().

13. Current Limitations

- **Authentication Security:** Passwords are currently stored as plaintext strings in the database rather than hashed with salt (bcrypt/Argon2). JWT tokens are not yet integrated.
- **Automated Test Coverage:** While manual browser and API testing confirm all features work, automated pytest suites under tests/ are unpopulated.
- **Database Engine Dualism:** The development environment runs on SQLite (heart_disease.db) while production scripts target PostgreSQL. Automated Alembic migrations are pending.
- **Evaluation Figure Exports:** ROC curves, confusion matrices, and SHAP summary plots are currently rendered in code; static figure exports under reports/figures/ are pending.
- **Educational System Framing:** HeartX is strictly an educational clinical decision support prototype and is not clinically certified for medical diagnosis.

14. Remaining Work and Prioritized Checklist

The remaining tasks before final academic presentation and project submission are prioritized below:

- **Priority 1 (Security):** Implement password hashing with passlib/bcrypt; integrate JWT token middleware in FastAPI.
- **Priority 2 (Testing):** Author automated backend unit and integration tests using pytest and httpx in tests/.
- **Priority 3 (Visualizations):** Generate and export high-resolution PNG plots (ROC curve, confusion matrix, SHAP summary) to reports/figures/.
- **Priority 4 (Database Migration):** Migrate local SQLite to containerized PostgreSQL using Docker Compose and configure Alembic migrations.
- **Priority 5 (Documentation):** Capture screenshots from the running web application and insert them into the Section 8 placeholders.

15. Conclusion

The HeartX – Heart Disease Prediction System has successfully evolved from a conceptual architecture into a working, multi-tier clinical decision support web platform. The predictive engine is anchored by an optimized XGBoost model achieving verified benchmarks of 88.04% accuracy, 93.14% recall, and 0.9145 ROC-AUC, outperforming seven competing algorithmic baselines. Unlike isolated machine learning scripts, HeartX operates as a complete database-driven system featuring a 3NF normalized schema,

asynchronous audit logging, runtime TreeSHAP explainability, and an intuitive React interface for clinical workflows. Completing the upcoming priorities—password hashing, automated test suites, and PostgreSQL migration—will bring the project to full production readiness.

16. Previous Research and Work in This Field

Over the past two decades, extensive research has investigated machine learning and computational intelligence techniques to automate and improve the early risk stratification of coronary heart disease. Pioneering investigations by Detrano et al. (1989) on the Cleveland clinic cohort laid the foundational benchmark for algorithmic cardiac risk assessment, demonstrating that statistical logistic regression could achieve discriminative accuracy comparable to invasive fluoroscopy. Subsequent literature transitioned toward ensemble learning and non-linear classification models; studies by Mohan et al. (2019) and Repaka et al. (2019) demonstrated that decision tree ensembles, Support Vector Machines (SVM), and Hybrid Random Forests significantly outperformed traditional linear baselines in cross-validated classification accuracy on clinical tabular datasets. More recently, gradient boosted decision trees—notably XGBoost and LightGBM—have emerged as state-of-the-art benchmarks in biomedical tabular informatics due to their robustness against data sparsity, handling of skewed distributions, and capacity to capture complex non-linear feature interactions without aggressive dimensionality reduction.

However, existing literature exhibits two critical shortcomings: first, the vast majority of prior work treats models as "black-box" systems, failing to provide instance-level interpretability that clinical practitioners require to trust and act upon predictions. The emergence of game-theoretic explainability frameworks, particularly SHAP (SHapley Additive exPlanations) introduced by Lundberg and Lee (2017), has begun to bridge this divide by deriving exact Shapley values to identify personalized biometric risk contributors. Second, most published research remains confined to isolated, offline Jupyter notebooks evaluated on static CSV files, omitting the relational database architectures (such as normalized 3NF schemas and audit logging), transactional integrity, and authenticated web services essential for transitioning predictive algorithms into clinical decision support workflows. HeartX builds upon this prior empirical foundation by coupling an optimized, recall-prioritized gradient boosted pipeline with runtime TreeSHAP explainability, underpinned by an operational, multi-tier database-driven application.

B. Feature-Wise Completion Status Table

Subsystem	Feature Description	Implementation Status	Verification Artifact
Machine Learning	Custom Feature Engineering Pipeline	**Completed**	src/features/engineering.py

Machine Learning	Mutual Information Feature Selection	**Completed**	src/features/selection.py
Machine Learning	8-Model Empirical Benchmarking	**Completed**	reports/model_comparison.csv
Machine Learning	Serialized XGBoost Model Pipeline	**Completed**	models/final/optimized_xgboost.pkl
Machine Learning	Runtime TreeSHAP Risk Factor Derivation	**Completed**	backend/main.py (lines 187-259)
Backend	FastAPI REST API Endpoints (6 routes)	**Completed**	backend/main.py
Backend	Asynchronous Background Audit Logging	**Completed**	backend/main.py (lines 74-79)
Database	Normalized 3NF Relational Schema (7 tables)	**Completed**	backend/models/domain.py
Database	PostgreSQL Production DDL Script	**Completed**	database/schema.sql
Frontend	React 18 SPA (5 Core Views & Auth Guards)	**Completed**	frontend/src/App.jsx & pages/